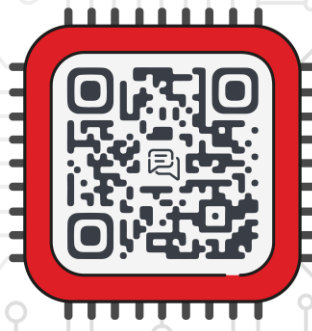




نظري

1

19

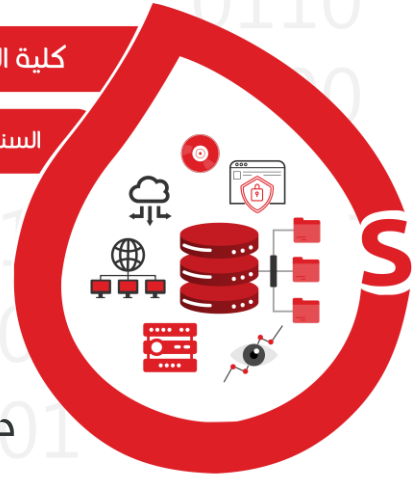
2850 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Implementation Of Databases

د. مادلين عبود



RB Informatcs; 9/10/2025

قواعد معطيات ٢

أهلاً وسهلاً في بداية رحلة جديدة في عالم قواعد البيانات، مقرر قواعد البيانات (Database 2)، وهو امتداد طبيعي لما تعلمناه في قواعد البيانات 1، ولكنه خطوة أعمق نحو فهم كيفية عمل أنظمة إدارة قواعد البيانات من الداخل. نتمنى لكم مقررًا مليئًا بالاستكشاف والفهم العميق والتطبيق الممتع.

تذكرة بمقرر قواعد البيانات 1:

في مقرر قواعد البيانات 1، تعرّفنا على المفاهيم الأساسية لبناء قاعدة البيانات من منظور المستخدم والمصمم، وركزنا على الجانب المنطقي والتحليلي (Logical Level)، حيث تعلمنا:

أهم المحاور السابقة:



- المفاهيم الأساسية لقواعد البيانات ومزاياها على الملفات التقليدية.
 - النمذجة باستخدام مخطط الكيانات والعلاقات (ERD).
 - التحويل من ERD إلى نموذج علائقي (Relational Model).
 - لغة SQL من حيث أوامر:
 1. إنشاء الجداول (CREATE TABLE).
 2. القيود (Constraints) مثل PRIMARY KEY, FOREIGN KEY, UNIQUE.
 3. الإدخال والتحديث والحذف (INSERT, UPDATE, DELETE).
 4. الاستعلامات (SELECT, JOIN, GROUP BY, HAVING, SUBQUERY).
 - فهم أساسيات التكامل بين الجداول، والتحكم في البيانات باستخدام SQL Server.
- بعبارة أخرى، تعلمنا كيف نتعامل مع البيانات على مستوى النموذج المنطقي والبرمجي.

مقدمة لمقرر قواعد البيانات 2:

في هذا المقرر، سننتقل إلى المستوى الفيزيائي (Physical Level) من قواعد البيانات، حيث سنتعرف على كيف تُخزن البيانات فعلاً داخل النظام، وكيف تتم معالجتها من قبل محركات قواعد البيانات. أي أننا سننتقل من كيف تكتب أوامر SQL إلى كيف ينفذ النظام هذه الأوامر فعلياً في الخلفية.

محتوى المادة

1. تخزين البيانات – Data Storage :

- كيف تخزن البيانات فعلياً داخل الملفات.

2. الفهرسة والتجزئة – Indexing and Hashing :

- طرق تسريع الوصول إلى البيانات.

3. معالجة الاستعلامات وتحسينها – Query Processing and Optimization :

- كيف يقوم النظام بتحليل الاستعلام (Parsing & Planning) .
- مراحل تنفيذ الاستعلام داخلياً.
- كيف يُقرر النظام "الخطة المثلى" لتنفيذ الاستعلام.
- استخدام الفهارس لتحسين الأداء.

4. إدارة العمليات (العمليات) – Transaction Management :

- مفهوم المعاملة (Transaction) وأهميتها في الحفاظ على سلامة البيانات.

5. التحكم في التوازي – Concurrency Control :

- كيف يمكن لعدة مستخدمين العمل على قاعدة بيانات واحدة في نفس الوقت دون تعارض.

6. قواعد البيانات الموزعة – Distributed Databases :

- المفهوم العام لقواعد البيانات الموزعة.

مقرر قواعد البيانات 1 كان مدخلاً للتعامل مع البيانات من الخارج. بينما مقرر قواعد البيانات 2 سيكشف لك كيف تُدار هذه البيانات من الداخل. سنتعلم في هذا المقرر كيف يفكر محرك قاعدة البيانات نفسه — كيف يخزن، كيف يبحث، كيف يُحسن، وكيف يحافظ على الأمان والتكامل حتى في وجود آلاف المستخدمين في الوقت نفسه.

العظمة في القيادة أن ترفع من حولك

لا أن تتعالى عليهم

والآن دعونا نبدأ في محتوى المحاضرة:

قاعدة المعطيات

قاعدة المعطيات: هي مجموعة مترابطة من البيانات (Data) تمثل جانباً محدداً من العالم الحقيقي، وتستخدم لغرض محدد من قبل مجموعة من المستخدمين المهتمين بها.

بمعنى آخر:

- البيانات ليست عشوائية، بل مرتبطة ومنظمة.
- كل قاعدة بيانات تعكس جانباً واقعياً (مثل الجامعة، البنك، الشركة).
- وتستخدم من قبل أشخاص أو برامج لخدمة هدف معين (كإدارة الطلاب، أو تتبع الحسابات، إلخ).



ولكن هل كل مجموعة من المعطيات المترابطة يمكن اعتبارها قاعدة معطيات؟
الإجابة: ليس بالضرورة.

مثلاً، جدول Excel فيه بيانات عن طلاب الجامعة ليس قاعدة بيانات حقيقية، ما لم:

- تنظم البيانات وفق نموذج علائقي أو هيكلي واضح.
- تدار بواسطة نظام إدارة قواعد البيانات (DBMS).
- تراعى فيها شروط التكامل، الأمان، والعلاقات بين الجداول.

ومن هذا المنطلق يمكن أن نستنتج **خصائص قاعدة المعطيات:**

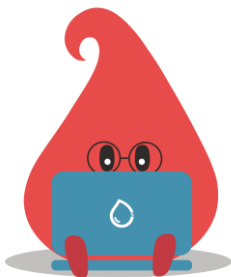
لكي نطلق على مجموعة من البيانات اسم **قاعدة البيانات** يجب أن تتوفر فيها الخصائص التالية:

- تمثيل جزء من العالم الحقيقي (Real-world mini-world):
أي أن البيانات تعبر عن واقع معين: طلاب، دورات، حسابات ...

- تحتوي على بيانات متلائمة ومنظمة (Integrated data):
أي لا يوجد تناقض بين الجداول، وكل معلومة لها موقع محدد.

- موجهة لهدف محدد (Purpose-oriented):
مثل نظام تسجيل طلاب، أو مخزون، أو نظام رواتب.

- تدار يدوياً أو آلياً باستخدام برنامج DBMS:
مثل MySQL, Oracle, SQL Server.



أمثلة توضيحية:

الجدول التالية:

الطلاب (Students):

name	st-number	class	major
Smith	17	1	CS
Brown	8	2	CS

المواد (Courses):

c-name	c-number	credit-hours	department
DataStruc	cp2001	3	IT
DataBase	cp3020	3	IT

المتطلبات السابقة (Prerequisite):

c-number	pre-number
cp2001	cp1200
cp3020	cp2001

- هذه الجداول مترابطة (العلاقة بين المواد والمتطلبات السابقة).
- تمثل جزءاً من العالم الحقيقي (نظام الجامعة).
- تخدم غرضاً واضحاً (إدارة التسجيل الأكاديمي).

لذا، هذا النموذج يُعتبر قاعدة بيانات متكاملة.

نظام إدارة قواعد البيانات:

أهم الخصائص التي تميز نظام إدارة قواعد المعطيات (DBMS) عن طرق تخزين البيانات التقليدية (مثل الملفات النصية أو جداول Excel):

1. وجود وصف ذاتي لقاعدة المعطيات:

المعنى:

قاعدة البيانات في أنظمة DBMS تمتلك وصفاً ذاتياً (Self-Describing Nature)، أي أنها لا تحتوي فقط على البيانات نفسها،

بل تحتوي أيضاً على بيانات تصف هذه البيانات.

المقصود بهذا الوصف:

- تعريف الجداول وأسماء الأعمدة.
- أنواع البيانات (رقم، نص، تاريخ...).
- العلاقات بين الجداول (مفاتيح أساسية وأجنبية).
- القيود (Constraints).



■ النتيجة:

يمكن للنظام أن يفهم بنية البيانات دون الحاجة إلى معرفة مسبقة من المبرمج بكل التفاصيل، لأنها مخزنة داخلياً في القاعدة نفسها.

2. Meta-Data البيانات الوصفية:

■ المعنى:

ال **Meta-Data** هي تلك المعلومات التي تصف بنية قاعدة البيانات، أي أنها بيانات عن البيانات.

■ مثال:

إذا كانت لديك قاعدة بيانات تحتوي على جدول Student، فإن ال Meta-Data تخبر النظام أن:

1. اسم الجدول: Student

2. الأعمدة: id, name, age

3. نوع id: عدد صحيح

4. نوع name: نص

5. نوع age: عدد

6. id هو المفتاح الأساسي

■ النتيجة:

بفضل وجود Meta-Data، يمكن للنظام التعامل مع أي قاعدة بيانات جديدة بنفس الطريقة دون تعديل الكود، لأن البنية موصوفة داخلياً.

3. العزل بين البرامج والبيانات (Data Independence):

■ المعنى:

في الأنظمة القديمة (نظام الملفات)، كان كل برنامج مرتبطاً مباشرة بطريقة تخزين البيانات،

أي أن أي تعديل في بنية الملف كان يتطلب تعديل البرنامج نفسه.

أما في أنظمة قواعد البيانات، فهناك **فصل تام** بين البيانات (Data) والبرامج (Programs) التي تستخدمها.

■ النتيجة:

1. يمكن تعديل بنية الجداول أو أنواع الأعمدة دون الحاجة إلى تغيير البرامج.

2. هذا يُسمى بـ Data Independence أي **استقلالية البيانات عن البرامج**.

أهمية هذه الميزة:

تجعل النظام أكثر مرونة وقابلية للصيانة وقابلية للتوسع.

الإنسان الناجح لا يبحث عن الأعذار، بل عن الحلول

4. التعامل مع أكثر من منظور للقاعدة:

■ المعنى:

يمكن لكل نوع من المستخدمين أن يرى قاعدة البيانات من **منظور مختلف** (View) يناسب احتياجاته. أي يمكن إنشاء **وجهات نظر** (Views) متعددة للبيانات نفسها.

■ مثال:

1. قسم الإدارة يرى فقط أسماء الموظفين ورواتبهم.
 2. قسم الموارد البشرية يرى العناوين وأرقام الهواتف.
 3. المدير يرى ملخصاً عاماً لجميع الأقسام.
- كل هؤلاء يتعاملون مع قاعدة واحدة لكن من زوايا مختلفة.

■ النتيجة:

1. سهولة في تخصيص البيانات لكل مستخدم.
2. حماية للبيانات الحساسة.
3. مرونة في تصميم الأنظمة الكبيرة.

5. مشاركة البيانات بين عدة مستخدمين ومعالجة المعاملات:

■ المعنى:

قاعدة البيانات يمكن أن تستخدم من قبل **عدة مستخدمين في نفس الوقت**، ويقوم النظام بإدارة هذه العمليات **دون تعارض أو فقدان للبيانات**.

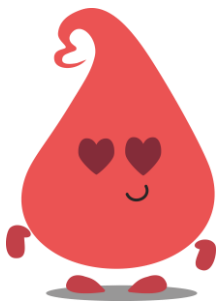
■ المقصود ب **معالجة المناقلات** (Transaction Processing):

كل عملية (مثل تحويل مبلغ أو تسجيل طالب) تعتبر معاملة (Transaction) ويضمن النظام أن تتم بالكامل أو لا تتم إطلاقاً (Atomicity).

■ النتيجة:

1. يستطيع عدة مستخدمين العمل في نفس الوقت بأمان.
2. لا تتداخل عملياتهم (Concurrency Control).
3. تحفظ سلامة البيانات (Data Integrity).

■ باختصار:



الشرح المختصر

الميزة

قاعدة البيانات تحتوي على معلومات تصف بنيتها الداخلية.	وصف ذاتي للبيانات
بيانات تصف بيانات أخرى (تعريف الجداول والأعمدة والعلاقات).	Meta-Data
تعديل القاعدة لا يتطلب تعديل البرامج.	فصل بين البرامج والبيانات
يمكن إنشاء Views مختلفة حسب نوع المستخدم.	تعدد وجهات النظر
السماح بعدة مستخدمين بالعمل المتزامن بشكل آمن ومنظم.	مشاركة البيانات

فائدة أرييسزية

1. Atomicity الذرية (خاصية التجزؤ أو الكلية):

■ المعنى:

ال Atomicity تعني أن العملية (Transaction) تنفذ ككتلة واحدة: إما تنفذ بالكامل، أو يلغى تنفيذها بالكامل، ولا يمكن تنفيذ جزء منها فقط.

■ مثال واقعي:

في بنك:

تريد تحويل 1000 ليرة من حساب (أ) إلى حساب (ب)

العملية تتكون من خطوتين:

1. خصم 1000 من حساب (أ).

2. إضافة 1000 إلى حساب (ب).

إذا حدث خطأ بعد الخطوة الأولى، يجب التراجع عن الخصم (Rollback)، حتى لا يفقد العميل ماله دون أن يصل للطرف الآخر.

أي أن المعاملة إما كلها أو لا شيء (All or Nothing).

2. Concurrency Control التحكم في التوازي:

■ المعنى:

في قواعد البيانات الكبيرة، يعمل عدة مستخدمين في الوقت نفسه على نفس الجداول. ال Concurrency Control هو النظام الذي يمنع تضارب العمليات بينهم.

■ مثال:

1. موظفان يحاولان تعديل نفس سجل الطالب في الوقت نفسه.

2. مستخدم يقوم بقراءة بيانات أثناء أن مستخدماً آخر يقوم بتعديلها.

يقوم النظام باستخدام آليات القفل (Locks) والطابع الزمنية (Timestamps)

لضمان أن التعديلات لا تتداخل وتبقى البيانات متناسقة.

الهدف: السماح بالتوازي في الأداء، بدون فقدان أو تضارب في البيانات.

3. Data Integrity سلامة البيانات:

■ المعنى:

هي مجموعة القواعد التي تضمن أن البيانات تبقى صحيحة ومنطقية ومتسقة في كل الأوقات.

■ أنواع سلامة البيانات:

1. Entity Integrity سلامة الكيان:

كل صف في الجدول يجب أن يُعرّف بمفتاح أساسي (Primary Key) غير مكرر وغير فارغ.

2. Referential Integrity سلامة العلاقات:

إذا كان هناك مفتاح أجنبي (Foreign Key) يشير إلى جدول آخر، فيجب أن تكون القيمة المشار إليها موجودة فعلياً في الجدول المرتبط.

3. Domain Integrity سلامة المجال:

القيم في العمود يجب أن تكون من نوع صحيح ومسموح به (مثل: العمر ≤ 0).

■ مثال:

1. لا يمكن إدخال طالب بلا رقم جامعي.
2. لا يمكن أن يشير سجل تسجيل إلى مادة غير موجودة في جدول المواد.

الهدف: منع الأخطاء المنطقية التي تجعل البيانات غير موثوقة.

4. Data Independence استقلالية البيانات:

■ المعنى:

قد تتغير بنية قاعدة البيانات (مثل تغيير اسم الجدول أو نوع العمود)، لكن البرامج التي تتعامل معها لا تحتاج إلى تعديل.

هذا يعني أن البيانات مستقلة عن البرامج.

■ مثال:

لو أضفنا عموداً جديداً إلى جدول Student، لن يتأثر النظام طالما لم يستخدم هذا العمود في الاستعلامات.

5. Data Security أمان البيانات:

■ المعنى:

هي حماية البيانات من الوصول غير المصرح به، ومن التعديل أو الحذف أو السرقة.

■ الآليات:

1. صلاحيات المستخدمين (Users & Roles).

2. كلمات مرور وتشفير.

3. تحديد من يمكنه القراءة أو التعديل أو الحذف.

6. Transaction المعاملة:

■ المعنى:

هي مجموعة من العمليات (مثل إدخال، تعديل، حذف)، تُنفذ كوحدة واحدة تحقق خصائص ACID.

■ خصائص ACID الأربع:

الخاصية	المعنى
A - Atomicity	كل العملية أو لا شيء
C - Consistency	تحافظ على تكامل البيانات
I - Isolation	تنفيذ المعاملات بشكل معزول عن بعضها
D - Durability	تبقى النتائج حتى بعد فشل النظام

7. MetaData البيانات الوصفية:

■ المعنى:

هي بيانات تصف بيانات أخرى

أي أنها تشرح بنية قاعدة البيانات: الجداول، الأعمدة، الأنواع، القيود.

تسمح للنظام بفهم قاعدة البيانات دون الحاجة لمبرمج يعرفها يدويًا.

8. View العرض (المنظور):

■ المعنى:

هي جدول افتراضي (افتراضي فقط) يُنشأ من استعلام

يُستخدم لإظهار جزء من البيانات أو دمج عدة جداول.

■ المثال:

```
CREATE VIEW StudentInfo AS
SELECT name, department From Students;
```

المستخدم يرى الجدول "StudentInfo"،

لكنه في الحقيقة نتيجة استعلام من الجدول الأصلي.

9. DBMS نظام إدارة قواعد المعطيات:

■ هو البرنامج الذي يدير كل ما سبق: ينظم، يخزن، يؤمن، ويسترجع البيانات.

■ مثال:

1. SQL Server

2. Oracle

3. MySQL

4. PostgreSQL

نهاية الفائدة الأريسيزية

الوظائف الأساسية لنظام إدارة قواعد المعطيات

الوظيفة	الشرح
تعريف البيانات (Data Definition)	إنشاء الجداول وتحديد أنواع الأعمدة والقيود باستخدام DDL (CREATE, ALTER, DROP).
معالجة البيانات (Data Manipulation)	إدخال، تحديث، وحذف البيانات باستخدام DML (INSERT, UPDATE, DELETE).
استرجاع البيانات (Data Retrieval)	تنفيذ استعلامات SQL للحصول على المعلومات المطلوبة (SELECT).
التحكم في التوازي (Concurrency Control)	ضمان أن المستخدمين المتعددين يمكنهم العمل دون تعارض.
الحماية والأمان (Security)	التحكم في صلاحيات المستخدمين (من يمكنه القراءة، التعديل، إلخ).
الاسترجاع بعد الفشل (Recovery)	استعادة البيانات في حال حدوث أعطال أو انقطاع مفاجئ.
التكامل والاعتمادية (Integrity)	ضمان أن العلاقات بين الجداول تبقى صحيحة (عبر المفاتيح الأساسية والأجنبية).

- من يتعامل مع قواعد المعطيات ؟
يوجد فئات مختلفة يمكن أن تتفاعل مع قاعدة البيانات، ولكل فئة دور محدد في تصميمها أو استخدامها.

الفئات الرئيسية:

الدور	الفئة
مسؤول عن إدارة النظام، ضبط الأمان، الصلاحيات، النسخ الاحتياطي، مراقبة الأداء.	Data Administrator (DBA)
يصممون الجداول والعلاقات، ويحددون نوع البيانات والقيود (المخطط المنطقي والفيزيائي).	Database Designers
يستخدمون قاعدة البيانات لإدخال البيانات أو الاستعلام عنها.	End Users
محلو النظام ومبرمجو التطبيقات الذين يطورون البرامج التي تتفاعل مع قاعدة البيانات.	System Analyst & Application Programmer

اليوم يُطلق على محلل النظام ومبرمج التطبيق اسم مهندس البرمجيات (Software Engineer) لدمج المهام التحليلية والبرمجية معاً.

- فئات إضافية من مستخدمي قواعد المعطيات (إلى جانب المستخدمين الأساسيين، هناك فئات أخرى لها أدوار داعمة ومهمة في بيئة قاعدة البيانات):

المهام	الفئة
يصممون ويبنون نظام DBMS نفسه، وليس فقط قاعدة البيانات.	Database System Designers & Implementers
يطورون أدوات مساعدة مثل أدوات الفهرسة أو مراقبة الأداء.	Tool Developers
يشغلون النظام يوميًا ويتأكدون من سلامة الأجهزة والبرامج والاتصال.	Maintenance & Operations Staff

فوائد استخدام نظم إدارة قواعد المعطيات

1. التحكم بالتكرار: منع تكرار نفس البيانات في أكثر من مكان.
2. تحقيق الأمان: تحديد صلاحيات الوصول للمستخدمين.
3. تخزين آمن ومنظم للبيانات والبرامج.
4. إمكانية الاستنتاج: استخراج معلومات جديدة من بيانات موجودة.
5. تقديم واجهات متعددة للتفاعل مع النظام.
6. تمثيل العلاقات المعقدة بين البيانات.
7. التحقق من شروط التكامل مثل المفاتيح.
8. دعم النسخ الاحتياطي والاسترجاع التلقائي backup & recovery.

مزايا استخدام منهج (منحى) قواعد المعطيات:

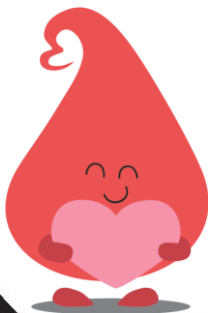
يؤدي اعتماد DBMS إلى تحسينات تنظيمية وتطبيقية، منها:

- توحيد المعايير بين الأقسام لتسهيل التعاون
- اختزال زمن تطوير الأنظمة لأن البيانات مركزية.
- المرونة في تعديل بنية القاعدة دون التأثير على التطبيقات.
- الوصول المباشر إلى البيانات الحديثة (Up to date).
- الاقتصاد في العمليات بتقليل الوقت والجهد في الاستعلامات.

متى نستخدم DBMS؟ ومتى لا نحتاجه؟

الاستخدام المثالي:

- عندما تكون البيانات كبيرة، متغيرة، ومشاركة بين عدة مستخدمين.
- عند الحاجة إلى الأمان، التكامل، والتعامل المتزامن.



متطلبات استخدامه:

- يحتاج إلى استثمار في الأجهزة، البرمجيات، والتدريب.
- يستهلك وقتًا في الإعداد والتشغيل.

الحالات غير المناسبة:

- عندما تكون البيانات بسيطة جدًا وثابتة.
- في تطبيقات وحيدة المستخدم أو أنظمة الزمن الحقيقي (Real-Time) التي تحتاج سرعة دون طبقة DBMS.

الاستخدام المثالي ل DBMS:

عندما تكون البيانات كبيرة، متغيرة، ومشاركة بين عدة مستخدمين:

1. البيانات الكبيرة (Large Data):

- عندما تحتوي المؤسسة على آلاف أو ملايين السجلات (مثل بيانات طلاب جامعة، حسابات بنك، أو زبائن شركة). في هذه الحالة، لا يمكن الاعتماد على ملفات بسيطة، لأن:
- صعوبة في البحث والتحديث.
 - احتمال التكرار أو الضياع.
 - بطء في الوصول.

2. البيانات المتغيرة (Dynamic Data):

- أي البيانات التي تتغير باستمرار (إضافة، تعديل، حذف).
مثال: الرصيد البنكي، جدول الدروس، الطلبات اليومية.

3. البيانات المشتركة (Shared Data):

- أي أن عدة مستخدمين أو تطبيقات يعتمدون على نفس البيانات.
- موظف يضيف بيانات.
 - مدير يراجعها.
 - برنامج آخر يحللها.
- كلهم يستخدمون نفس قاعدة البيانات في الوقت نفسه.

عند الحاجة إلى الأمان، التكامل، والتعامل المتزامن:

1. الأمان (Security):

- حماية البيانات من الوصول غير المصرح به عبر كلمات مرور وصلاحيات.
مثال: المستخدم العادي لا يرى الرواتب، والمدير فقط يمكنه التعديل.



2. التكامل (Integrity):

ضمان أن العلاقات بين الجداول تبقى صحيحة (مثل أن الطالب المسجل في مادة يجب أن يكون موجوداً في جدول الطلاب أصلاً).

3. التعامل المتزامن (Concurrent Access):

أي إمكانية أن يعمل أكثر من مستخدم في نفس الوقت دون تضارب أو تلف في البيانات.

- موظفان يعدّان جدول المبيعات معاً.

- أحدهما يضيف والآخر يحذف.

النظام يتحكم بالتوازي (Concurrency Control) لمنع التداخل.

متطلبات استخدام DBMS:

يحتاج إلى استثمار في الأجهزة، البرمجيات، والتدريب:

أنظمة DBMS ليست مجانية أو بسيطة دائماً، وتتطلب:

- خادم قوي (Server) لتخزين البيانات وإدارة المستخدمين.

- رخصة استخدام (مثل: SQL Server, Oracle).

- تدريب للمبرمجين والمستخدمين على استخدام النظام.

أي أنها مكلفة في البداية لكنها فعالة على المدى الطويل.

يستهلك وقتاً في الإعداد والتشغيل:

قبل أن تبدأ قاعدة البيانات بالعمل، يجب:

- تحليل البيانات وتصميم الجداول والعلاقات.

- إعداد صلاحيات المستخدمين.

- اختبار الأداء والنسخ الاحتياطي.

أي أن البداية تحتاج جهداً، لكنها تضمن نظاماً مستقرّاً وآمناً بعد التشغيل.



الحالات غير المناسبة لاستخدام DBMS:

عندما تكون البيانات بسيطة جداً وثابتة:

إذا كانت البيانات صغيرة ولا تتغير غالباً، مثل:

- ملف أسماء طلاب دفعة واحدة.

- قائمة أسعار ثابتة لا تتغير.

- إعدادات بسيطة لتطبيق.

في هذه الحالات، لا داعي لاستخدام DBMS، لأن ملف Excel أو JSON يكفي.

في تطبيقات وحيدة المستخدم أو أنظمة الزمن الحقيقي (Real-Time Systems):

أنظمة الزمن الحقيقي: هي الأنظمة التي يجب أن تستجيب فوراً (في أجزاء من الثانية) لأي حدث يحدث، ولا تحمل أي تأخير في المعالجة، أمثلة:

- نظام التحكم في المكابح داخل السيارات.
- نظام توجيه الطائرات أو الأقمار الصناعية.
- مراقبة العمليات الطبية في غرفة العمليات.

السبب في عدم استخدام DBMS هنا:

- أنظمة DBMS تضيف طبقة وسيطة (Layer) بين البيانات والبرنامج، مما يسبب تأخيراً صغيراً أثناء المعالجة.
 - هذا التأخير مقبول في الأنظمة الإدارية، لكنه غير مقبول في الأنظمة التي تتطلب استجابة فورية.
- لذلك، هذه الأنظمة تستخدم تخزين مباشر في الذاكرة (In-memory) أو هياكل بيانات خاصة لتحقيق سرعة فورية.

مفهوم طبقة DBMS (DBMS Layer):

عندما نقول إن DBMS يشكل "طبقة" إضافية، نقصد أن التطبيق لا يتعامل مباشرة مع البيانات المخزنة على القرص، بل يتعامل مع DBMS، وهو الذي يدير القراءة والكتابة.

- الفائدة: الأمان والتنظيم.
- العيب: زيادة طفيفة في زمن التنفيذ (Latency).

مخطط القاعدة (database schema)

المصطلح	المعنى
Database schema	الهيكل الثابت للقاعدة: الجداول، الأعمدة، العلاقات، القيود.
Database Instance (State)	الحالة الحالية للبيانات، تتغير مع كل عملية إدخال أو حذف أو تحديث

لغات DBMS:

- DDL (Data Definition Language): إنشاء الجداول والمخططات.
- DML (Data Manipulation Language): إدخال، تعديل، حذف البيانات.

لا تنتظر اللحظة المناسبة

بل اصنعها بعملك وإصرارك



المفاهيم في نظم قواعد المعطيات

يتناول الأساس النظري لنماذج البيانات (Data Models) ، أي الطرق المختلفة لتمثيل البيانات والعلاقات بينها.
الأنواع الأساسية:

- Conceptual Models (ER Model):
تصف الكيانات (Entities) والعلاقات (Relationships) والسمات (Attributes).
- Logical Models (Relational Model):
تترجم إلى جداول وأعمدة ومفاتيح.
- Physical Models:
تصف كيفية تخزين البيانات فعلياً في الملفات والصفحات.

نموذج الكيان والعلاقة (ER Model):

نموذج الكيان والعلاقة يصف البنية المفاهيمية للبيانات قبل تنفيذها في النظام.

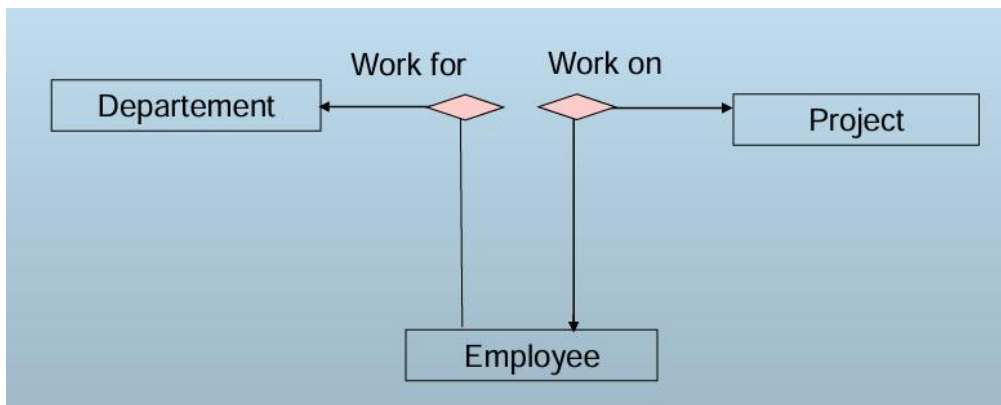
مثال تطبيقي:

شركة تحتوي على:

- عدة أقسام (Department).
- عدة مشاريع (Project).
- عدة موظفين (Employee).
- علاقات تربط بينهم مثل:
 1. كل قسم يدير عدة مشاريع.
 2. كل موظف يعمل في مشروع أو أكثر.
 3. كل قسم له مدير واحد.



هذا يُرسم في مخطط ERD (Entity Relationship Diagram) لتمثيل الكيانات والعلاقات بينهما.



EERD (Enhanced Entity Relationship Diagram):

ال EERD هو تطوير لنموذج ER بإضافة مفاهيم من البرمجة الكائنية (Object-Oriented)، مثال:

المفهوم	الشرح
الوراثة (Inheritance)	علاقة صف أب (Superclass) وصف ابن (Subclass) كما في البرمجة.
تخصيص (Specialization)	تقسيم كيان عام إلى كيانات فرعية خاصة (مثل: موظف → مهندس، مدير).
تعميم (Generalization)	عكس السابقة، دمج كيانات فرعية ضمن كيان عام.
الجمع (Union)	جمع كيانات مختلفة في كيان واحد مشترك.
التجميع (Aggregation)	علاقة كيان مع علاقة أخرى (علاقة من مستوى أعلى).

أمثلة من المحاضرة عن الوراثة والتعميم والتخصيص:

1. الوراثة:

لدينا كيان عام موظف (Employee)، له كيانات فرعية:

1. مدير Manager.

2. مهندس Engineer.

3. سكرتير Secretary.

كل من هذه الكائنات يرث الصفات العامة مثل name, address, salary ويضيف صفات خاصة، مثل:

1. Manager: عدد الموظفين الذين يديرهم.

2. Engineer: نوع الاختصاص.

3. Secretary: سرعة الطباعة.



2. التخصيص:

هي عملية تقسيم كيان عام إلى كيانات فرعية أكثر تحديداً،

بناءً على خصائص مميزة أو أدوار مختلفة.

أي أننا نخصّص الكيان العام حسب خصائص معينة.

كيان عام موظف Employee:

خصائص التمييز: نوع العمل.

النتيجة:

1. Engineer

2. Technician

3. Administrator

الهدف من التخصيص: تنظيم البيانات حسب طبيعة الدور أو الصفة.

3. التعميم:

■ هي عكس Specialization، أي دمج كيانات فرعية متشابهة في كيان عام واحد لأنها تشترك في صفات كثيرة.

■ أي أننا نَعَمِّم الخصائص المشتركة في كيان واحد.

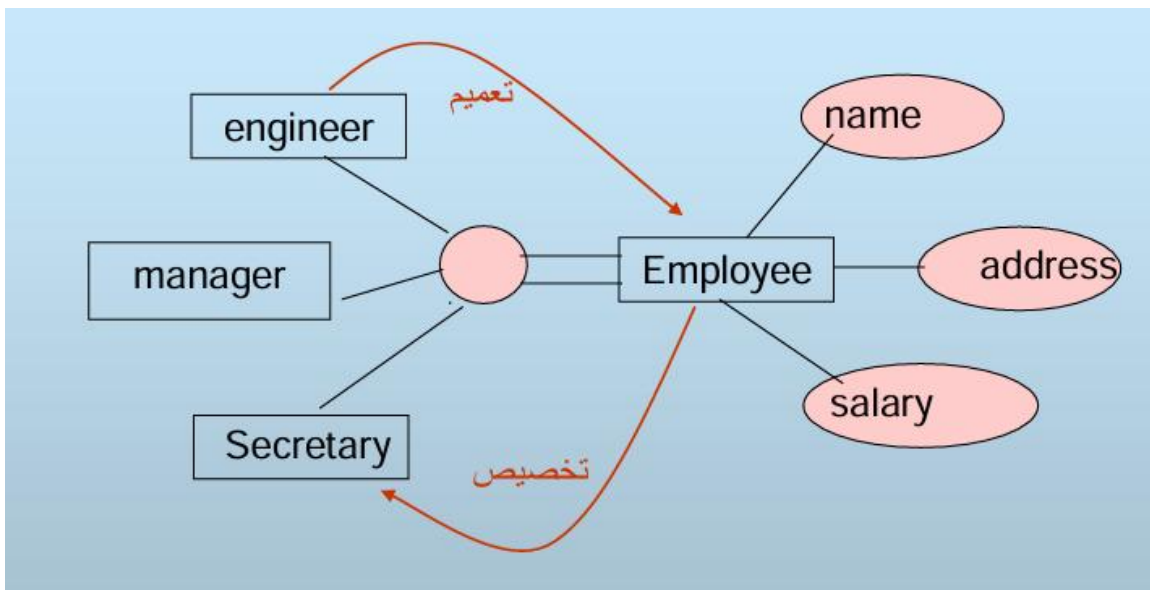
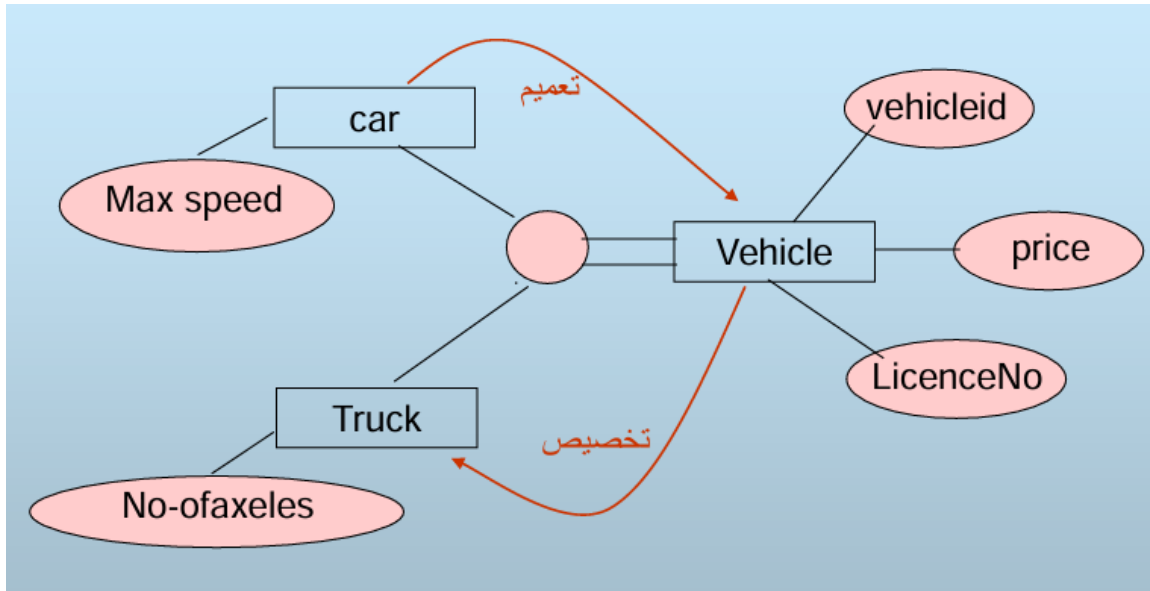
■ لدينا كيانات:

1. سيارة Car.

2. شاحنة Truck.

نلاحظ أنهما يشتركان في صفات مثل Price, Model, Color.

فننشئ كياناً عاماً اسمه مركبة Vehicle يحتوي على الخصائص المشتركة ويصبح الكيانان Car و Truck حالتين فرعيتين منه.

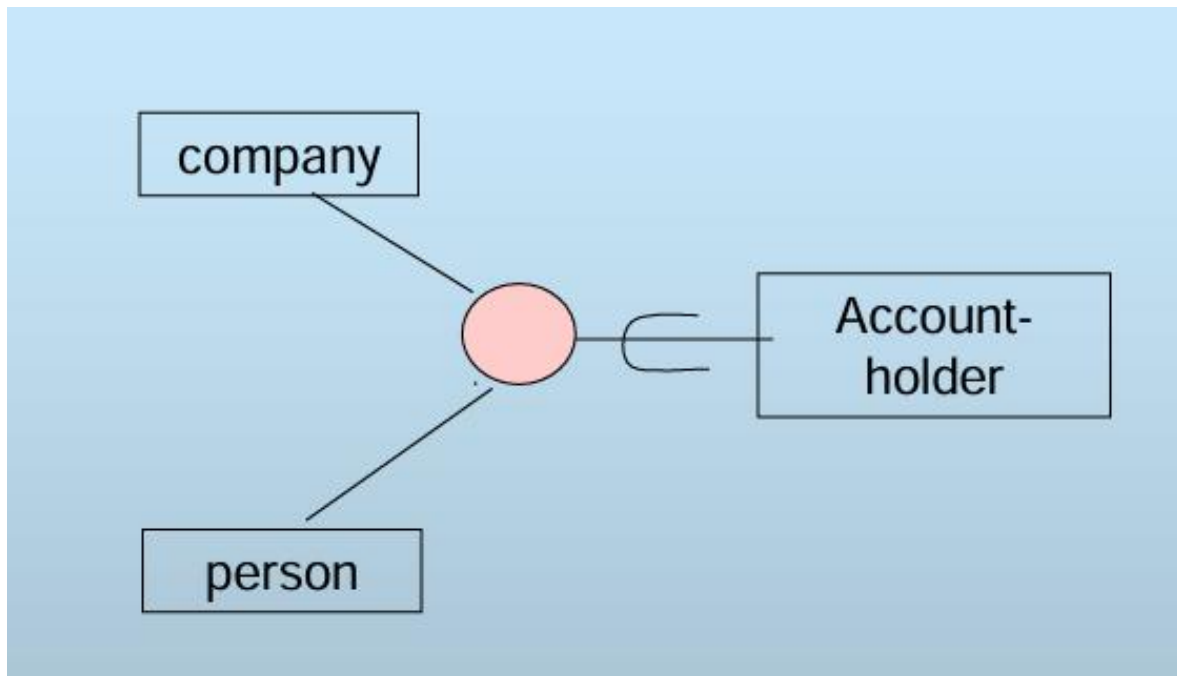


4. الاتحاد (الجمع):

- نستخدم عندما يكون هناك كيان مشترك يمثل عدة أنواع مختلفة من الكيانات، بحيث يمكن أن يرتبط هذا الكيان بأحدها فقط في كل مرة.
- أي أن الكيان المشترك يُمكن أن يكون **ناتج اتحاد** بين أكثر من كيان.

■ كيان مشترك صاحب حساب (AccountHolder):
يمكن أن يكون:

1. شخص Person.
 2. شركة Company.
- أي أن الحساب البنكي يمكن أن يخص شخصاً أو شركة.



5. التجميع:

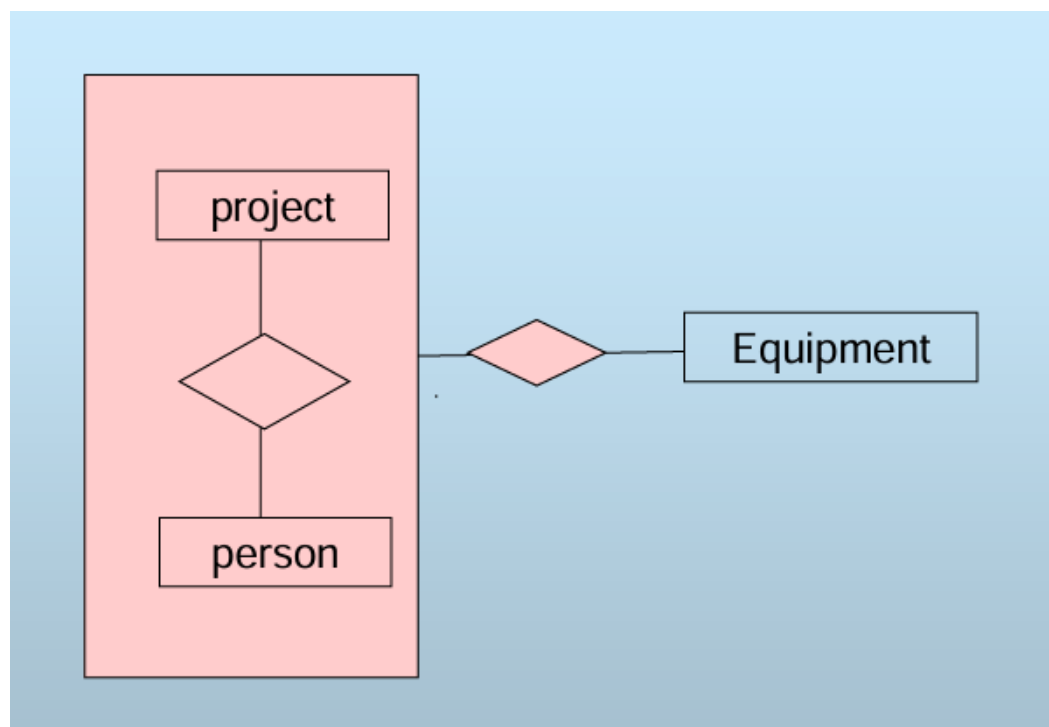
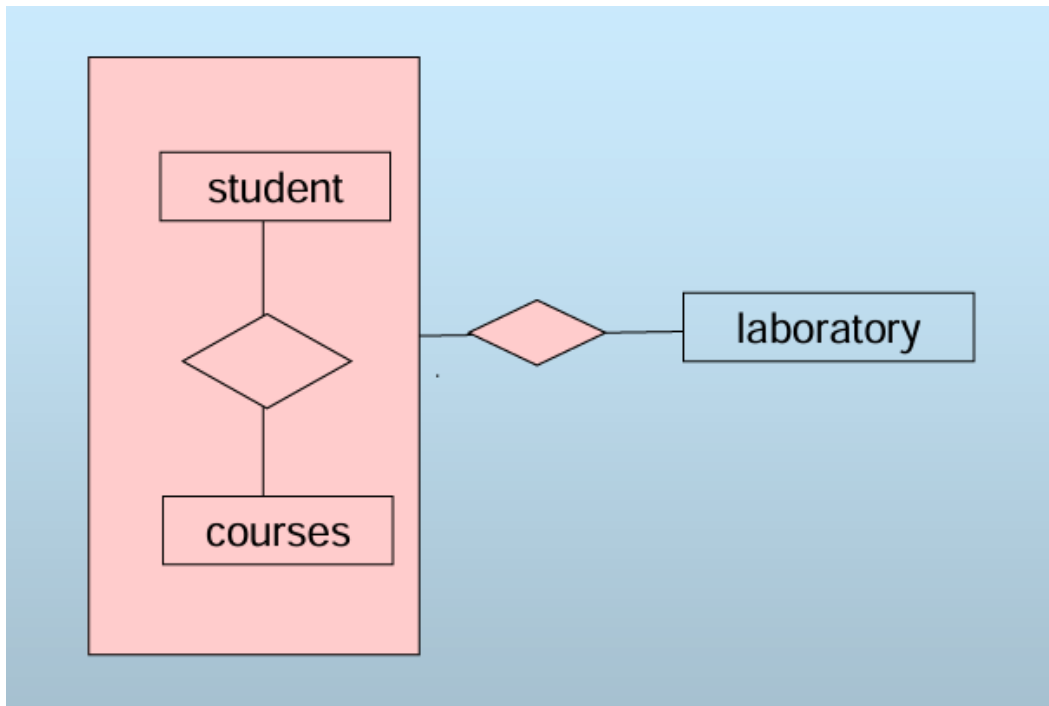
- هي علاقة من مستوى أعلى، تربط كياناً بعلاقة أخرى (أي علاقة تحتوي علاقة + كيان آخر).
- نستخدم عندما تكون العلاقة نفسها ذات معنى يمكن ربطه بكيانات أخرى.

■ لدينا علاقة :

Student – RegistersIn – Course

ونريد أن نضيف كيان مختبر Laboratory الذي يعتمد على هذه العلاقة (الطالب المسجل في مادة معينة يعمل في مختبر محدد).

إذاً العلاقة RegistersIn تُعامل ككيان جديد مرتبط ب Laboratory.



The End

لا تجعل خوفك من الفشل يمنعك من تجربة النجاح